

Измерение времени выполнения кода

Владислав Богданов

2026-06-16

Оглавление

1. На микроконтроллерах	1
1.1. ARM Cortex-M	1
1.1.1. DWT cycle counter(самый точный)	1
1.1.2. GPIO + Осциллограф / логический анализатор	1
1.1.3. SysTick или аппаратный таймер	1
2. На Linux (для драйверов и user-space кода)	2
2.1. Clock_gettime - стандарт POSIX	2
2.2. rdtsc - для x86 (если драйвер на ПК)	2
2.3. Профилировщики	2

1. На микроконтроллерах

1.1. ARM Cortex-M

1.1.1. DWT cycle counter(самый точный)

У cortex-M3/M4 есть регистр DWT_CYCCNT - 32-битный счетчик тактов ядра

```
// Включение счётчика (один раз при инициализации)
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
DWT->CYCCNT = 0;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;

// Измерение
uint32_t start = DWT->CYCCNT;
// ... твой код ...
uint32_t end = DWT->CYCCNT;
uint32_t cycles = end - start;

// Перевод в микросекунды при частоте ядра 72 МГц
float us = (float)cycles / 72.0f;
```



счётчик 32-битный, при 72 МГц это ~60 секунд. Для длительных замеров считай переполнения.

1.1.2. GPIO + Осциллограф / логический анализатор

Самый наглядный метод — дёргаешь пин до и после:

```
GPIO_SetBits(GPIOA, GPIO_Pin_5); // маркер начала
// ... измеряемый код ...
GPIO_ResetBits(GPIOA, GPIO_Pin_5); // маркер конца
```

На осциллографе видишь прямоугольный импульс — его ширина = время выполнения. Идеально для проверки циклов и прерываний.

1.1.3. SysTick или аппаратный таймер

Если DWT недоступен, используй SysTick:

```
uint32_t start = SysTick->VAL;
// ... код ...
uint32_t end = SysTick->VAL;
// SysTick считает вниз!
uint32_t ticks = (start > end) ? (start - end) :
```

```
(SysTick->LOAD - end + start);
```

2. На Linux (для драйверов и user-space кода)

2.1. Clock_gettime - стандарт POSIX

```
#include <time.h>

struct timespec ts_start, ts_end;
clock_gettime(CLOCK_MONOTONIC, &ts_start);
// ... код ...
clock_gettime(CLOCK_MONOTONIC, &ts_end);

long ns = (ts_end.tv_sec - ts_start.tv_sec) * 1000000000L
          + (ts_end.tv_nsec - ts_start.tv_nsec);
```

2.2. rdtsc - для x86 (если драйвер на ПК)

```
static inline unsigned long long rdtsc() {
    unsigned int lo, hi;
    __asm__ volatile ("rdtsc" : "=a" (lo), "=d" (hi));
    return ((unsigned long long)hi << 32) | lo;
}
```

2.3. Профилировщики

perf — perf stat ./prog, perf record + perf report

gprof — компилируй с -pg, смотри gmon.out

valgrind --tool=callgrind + KCachegrind — детальный граф вызовов